

Homework: Design Patterns + Memory Management & Garbage Collection

Deliverables (2 files):

1. **.py** — all of the code implementations and the experiment (not a notebook).
Include a `if __name__ == "__main__":` section that demonstrates your code and runs the experiment in the GC section.
2. **.pdf** — short written explanations + outputs + reasoning.

The code must be legible, a complete solution includes docstrings and basic error catching.

Both files must have the name and id of the student at the top.

Part 1: Design Patterns

For each question:

- Implement a design that fits the use-case below.
- Your design must be pattern-based, using something you learned in class.
- In the PDF, justify your choice of pattern, and where possible, what the drawbacks of at least one alternative design would be.

Q1 (20 pts):

You are given the following `DummyModel` class with a `predict(X)` method.

```
class DummyModel:

    def __init__(self):

        self.model = ''

    def predict(self, X):

        """ Return random 0 and 1 values for each sample in X """

        return np.random.randint(0, 2, len(X))
```

You want to evaluate its predictions using several possible evaluation methods (accuracy, F1, precision, etc.). Add this functionality.

Requirements:

1. Don't change the original `DummyModel` class.
2. You should be able to **switch the evaluation method at runtime** without changing the model class.
3. The evaluation logic must be **extensible**: adding a new metric should not require modifying existing code.
4. In `__main__`: Generate random data (`X`, `y_true`), each of size 100 samples, and demonstrate evaluating the model with at least two different metrics from `sklearn.metrics` using your pattern.

In the PDF:

Explain the reasoning behind your design, including why your chosen pattern supports modular evaluation behavior and why it is extensible.

Q2 (20 pts):

You are implementing a small system that tracks **current CPU usage** and allows multiple independent components to react whenever a new measurement arrives.

You must design an architecture that lets different components (“monitors”) automatically respond to CPU updates.

Scenario

You are asked to implement the following components:

- A class **CpuSensor** that periodically measures CPU usage (you may simulate values instead of reading real CPU stats).
- Several monitors that want to react whenever the CPU usage changes.
 1. **LoggingMonitor** holds an attribute “_usage_log” that is a dictionary, Everytime the CPU usage changes, it appends the new value to “_usage_log” with the timestamp as key. Has a method to print the log.
 2. **HighUsageWarningMonitor** If usage exceeds a threshold (e.g., 85 percent) it prints a warning message.
 3. **GcTriggerMonitor** Whenever the usage is above a certain threshold (e.g., 90 percent), it explicitly calls `gc.collect()` and prints the number of collected objects.

Requirements

1. **CpuSensor** must not know what the monitors do or how many exist.
2. Monitors must automatically be notified whenever the CPU usage changes.
3. You must allow monitors to be added and removed dynamically at runtime.
4. You must demonstrate your design in `__main__`:
 - Create a **CpuSensor**.
 - Attach the three different monitor types.
 - Simulate several CPU usage updates (e.g., random values or a predefined list).
 - Show that each monitor reacts appropriately.

In the PDF

Explain the following:

1. Justify your choice of pattern, refer to why it is better than `CpuSensor` directly calling the monitors.
2. How forgetting to detach monitors could cause memory growth.

Q3 (20 pts):

You want to create a unified logging interface that supports multiple implementations:

- `ConsoleLogger` – prints messages to console.
- `FileLogger` – appends messages to a file.
- `NullLogger` – does nothing (for disabling logging).

The input is a configuration dictionary with the keys “logger_type” and “path”, e.g.:

```
config = {"logger_type": "file", "path": "log1.txt"}
```

The output should be a logger object, including a method `log(input)`.

Requirements:

1. Implement a design that returns the appropriate logger object based on the config.
2. Your design must allow adding new logger types without modifying existing client code.
3. Demonstrate usage with at least two configurations in `__main__`, and in particular show that they work through the same interface.

In the PDF:

1. Assuming your design is not a simple list of “if”s- why is it better than that?
2. Explain why your architectural choice works well when scaling to additional logger types.

Part 2: Memory Management and Garbage Collection.

In this part you will run two experiments using two different implementations of the CPU-monitoring system from Q2. The goal is to compare how the two designs behave with respect to memory usage and garbage collection when many short-lived monitors are repeatedly added and discarded.

Q4(a): (10 pts)

Use your **original implementation from Q2** exactly as it is.

In that version, the CPU sensor keeps its monitors in a normal Python container (such as a list or set).

Write the following diagnostic function:

```
def accumulating_monitor_demo(n=10000, report_every=1000):  
  
    import gc  
  
    sensor = CpuSensor()  
  
    for i in range(n):  
  
        m = LoggingMonitor()  
  
        sensor.attach(m)  
  
        del m  
  
        if (i + 1) % report_every == 0:  
  
            print(f"[A] After {i+1} attachments → monitor count =  
{len(sensor._monitors)}")  
  
            print("GC counts:", gc.get_count())
```

Run this once in `__main__`.

In the PDF, report:

1. The printed counts at several (relevant) checkpoints.
 2. Whether the number of monitors appears to grow, stay stable, or shrink.
 3. Observations about how the GC counters behave.
 4. Why the objects created inside the loop do or do not get freed.
-

Q5(b): (15 pts)

Now create an **alternative version** of the sensor in which the internal storage of monitors is implemented **differently** from Q2.

This version should behave the same externally (same methods, same usage), but internally should not retain monitors indefinitely after the local variable is deleted.

Name this class something like:

```
class AlternativeCpuSensor:
```

```
    ...
```

This sensor should still:

- accept monitor objects via `attach`
- notify them during updates
- behave like the Q2 sensor from the perspective of calling code

But, unlike the Q2 version, it should naturally allow monitors to disappear when nothing else refers to them.

Then write a second experiment:

```
def vanishing_monitor_demo(n=10000, report_every=1000):  
  
    import gc  
  
    sensor = AlternativeCpuSensor()  
  
    for i in range(n):  
  
        m = LoggingMonitor()  
  
        sensor.attach(m)  
  
        del m  
  
        if (i + 1) % report_every == 0:  
  
            print(f"[B] After {i+1} attachments → monitor count =  
{len(sensor._monitors)}")  
  
            print("GC counts:", gc.get_count())
```

Run it after the first experiment so the comparison is obvious.

Q5(c): Compare the Two Designs (15 pts)

In the PDF, include:

1. The outputs of both experiments.
2. A comparison:
 - Does the monitor count grow in version A?
 - Does the monitor count stay low or disappear in version B?
 - How do the GC counters behave in both cases?

3. An explanation in your own words:

- Why monitors accumulate in the first design
- Why they do *not* accumulate in the second design
- What internal difference in the containers explains this behavior
- How this relates to Python's memory-management model

4. A brief practical takeaway (3–5 sentences):

- What should long-running systems do when managing short-lived observers or callbacks?